Université de Rouen Normandie

Département Informatique — Licence 3 Informatique

Rapport de Projet Annuel

Système Intelligent d'Aide au Triage des Radiographies Thoraciques

PneumoScan

Prototype Académique — Projet Annuel de Licence

Triage automatisé par apprentissage automatique (EfficientNet-B0 + SVM)
avec interface web, API REST, authentification JWT/RBAC
et tableau de bord BI

Étudiant : Anis Aissani

Binôme : Mohammed Elakef Zenagui

Encadrant : Prof C.Petitjean

Établissement : Université de Rouen Normandie

Formation : L3 Informatique

Année : 2025 / 2026

GitHub : `github.com/Anis-Aissani/Pneumonia-Detection-App`

Contents

Liste des tableaux **iv**

1 Généralités sur le Contexte de Travail et la Méthodologie de Développement **1**

1.1 Définition du Domaine de Projet 1

1.1.1 Application web 1

1.2 Contexte du Projet 2

1.3 Problématique et Solution 2

1.3.1 Problématique 2

1.3.2 Solution 2

1.4 Quelques définitions concernant le domaine 2

1.5 Étude Critique des Applications Existantes 3

1.5.1 Bricoraml 3

1.5.2 Exemples de systèmes IA médicaux 3

1.5.3 Fiverr 3

1.5.4 Outils académiques open source 3

1.5.5 TaskRabbit 4

1.5.6 Exemples de datasets médicaux publics 4

1.5.7 Maquettes 4

1.5.8 Comparaison 4

1.5.9 Conclusion 4

1.6 Méthodologie de Développement 5

1.6.1 Unified Process (UP) 5

1.6.2 Les types de diagrammes UML 5

1.7 Plateforme de Développement et Stack Technique 6

1.7.1 Technologies Utilisées 6

1.7.2 Choix Techniques 6

1.8 Objectifs et Périmètre de l'Application 8

1.9 Conclusion du Chapitre 8

2 Analyse et Spécification des Besoins 9

2.1 Introduction 9
2.2 Besoins Fonctionnels 9
2.2.1 Gestion des Utilisateurs 9
2.2.2 Gestion des sessions de travail 9
2.2.3 Interaction et Communication 10
2.2.4 Priorisation des cas critiques 10
2.2.5 Recherche et Découverte 11
2.2.6 Anonymisation des images 11
2.2.7 Système d'Évaluation et de Réputation 11
2.2.8 Interfaces de l'application (captures réelles) 12
2.3 Analyse Non Fonctionnelle 18
2.4 Acteurs et Rôles 18
2.4.1 Rôle de supervision (administrateur) 18
2.4.2 Rôle opérationnel (radiologue) 19
2.4.3 Administrateur 19
2.5 Acteurs et Cas d'Utilisation 19
2.5.1 Diagramme de cas d'utilisation 19
2.6 Description des Cas d'Utilisation 20
2.6.1 Pour les Utilisateurs 20
2.6.2 Publication, Recherche et Interaction 20
2.6.3 Paiement et Évaluation 21
2.7 Diagramme de cas d'utilisation 21
2.8 Conclusion du Chapitre 21

3 Conception et Réalisation 22

3.1 Introduction 22
3.2 Architecture Logicielle Refactorisée (Clean Code & SOLID) 22
3.3 Diagrammes des Séquences 23
3.3.1 Connexion 24
3.3.2 Inscription 24
3.3.3 Soumettre une radiographie 24
3.3.4 Consulter mes activités 25
3.3.5 Ouvrir une session de travail 25
3.3.6 Consulter la file de triage 26
3.3.7 Consulter le tableau de bord BI 26
3.3.8 Générer un rapport PDF 27
3.3.9 Diagramme de classe 27

3.3.10 Dictionnaire des données 29
3.3.11 Le modèle relationnel 30
3.3.12 Code SQL 30
3.4 Outils, Langages et Environnements de Développement 31
3.4.1 Langages de Programmation 32
3.4.2 Frameworks et Bibliothèques 32
3.4.3 Base de Données 32
3.4.4 Communication en Temps Réel 32
3.4.5 Déploiement et Hébergement 33
3.4.6 Stockage des fichiers 33
3.4.7 Environnements de Développement 34
3.5 Description des Résultats Attendus 34

Conclusion générale **35**

Résumé **36**

Liste des mots-clés et leurs définitions **37**

Bibliographie **38**

Acronyms **40**

Acronyms **41**

List of Tables

1.1 Comparaison des solutions existantes 4
1.2 Phases UP appliquées au projet 5
1.3 Stack technique du projet 6
1.4 Synthèse des approches candidates (jeu de test) 7
1.5 Périmètre fonctionnel du système 8

2.1 Besoins fonctionnels — Gestion des utilisateurs 9
2.2 Besoins fonctionnels — Gestion des sessions 10
2.3 Besoins fonctionnels — Analyse radiographique 10
2.4 Besoins non fonctionnels 18
2.5 Acteurs du système et leurs responsabilités 18
2.6 CU-01 — Se connecter 20
2.7 CU-02 — Soumettre une radiographie 20

3.1 Organisation modulaire du backend après refactorisation 23
3.2 Dictionnaire des données — Table `predictions` 29
3.3 Dictionnaire des données — Table `sessions` 30
3.4 Langages de programmation utilisés 32
3.5 Frameworks et bibliothèques 32
3.6 Configuration Docker Compose 33
3.7 Environnement de développement 34
3.8 Résultats attendus du système 34

1. Généralités sur le Contexte de Travail et la Méthodologie de Développement

1.1 Définition du Domaine de Projet

1.1.1 Application web

PneumoScan est une application web d'aide au triage des radiographies thoraciques. Elle s'inscrit dans le domaine de la **santé numérique** et de l'**optimisation des workflows médicaux** par l'intelligence artificielle.

La pneumonie représente l'une des principales causes de mortalité infectieuse dans le monde, avec environ 2,5 millions de décès annuels [16]. Son diagnostic repose sur l'analyse de clichés radiographiques thoraciques, tâche à fort volume dans les services de radiologie. PneumoScan propose d'automatiser l'attribution d'un niveau de priorité à chaque cliché soumis, afin d'optimiser l'ordre de lecture du radiologue.

Répartition du travail en binôme

Ce projet est réalisé en binôme. **[Mon apport]** couvre la conception et le développement de l'architecture logicielle complète : API REST, interface web, base de données, sécurité et déploiement. **[Binôme]** couvre l'entraînement et la validation des modèles de machine learning (descripteurs classiques et CNN). Mon travail applicatif s'appuie donc sur son artefact modèle : j'ai intégré son résultat dans l'application tout en comprenant l'ensemble de la chaîne, de l'entraînement jusqu'à l'usage clinique dans l'interface.

1.2 Contexte du Projet

Ce projet annuel de Licence Informatique (L3) est conduit à l'Université de Rouen Normandie durant l'année universitaire 2025–2026. Il répond à une double exigence : d'une part, démontrer la maîtrise des outils du génie logiciel (conception, développement, tests, déploiement) ; d'autre part, illustrer l'intégration d'un composant d'IA dans une architecture applicative professionnelle.

1.3 Problématique et Solution

1.3.1 Problématique

Dans un service de radiologie, les clichés thoraciques sont généralement traités dans l'ordre de leur arrivée (mode FIFO). Ce mode de fonctionnement expose à un risque clinique réel : un cas de pneumonie sévère peut attendre plusieurs heures avant d'être examiné si des examens moins urgents ont été soumis antérieurement.

> *« Comment concevoir un système logiciel intégrant un modèle de machine learning pré-entraîné pour assigner automatiquement un niveau de priorité clinique à chaque radiographie soumise, tout en garantissant la traçabilité, la sécurité des accès et la conformité aux exigences de confidentialité des données patients ? »*

1.3.2 Solution

La solution proposée est **PneumoScan**, un système web composé de :

- Une **API REST** (FastAPI, Python 3.11) qui orchestre le pipeline de traitement des images et expose les données. **[Mon apport]**
- Une **interface utilisateur** (Vue.js 3) permettant au radiologue de soumettre des clichés, visualiser les résultats et consulter la file de triage en temps réel. **[Mon apport]**
- Un **modèle de classification** (EfficientNet-B0 + SVM, avec fallback HOG+SVM) entraîné et validé sur le dataset Chest X-Ray de Kaggle [7]. **[Binôme]**

1.4 Quelques définitions concernant le domaine

Radiographie thoracique (RX) Examen d'imagerie médicale utilisant les rayons X pour visualiser les poumons, le cœur et les structures médiastinales.

Pneumonie Infection du parenchyme pulmonaire, visible sur un cliché RX par des opacités alvéolaires caractéristiques.

Triage clinique Processus d'évaluation et de classification des patients selon l'urgence de leur prise en charge médicale.

HOG Descripteur de features locales basé sur la distribution des orientations des gradients dans une image [2].

SVM Algorithme de classification supervisée cherchant l'hyperplan séparateur de marge maximale [1].

JWT Standard ouvert (RFC 7519) pour la transmission sécurisée d'informations entre parties sous forme de token signé [6].

OOD (Out-Of-Distribution) Images ne ressemblant pas aux données d'entraînement, pouvant provoquer des prédictions erronées à haute confiance.

1.5 Étude Critique des Applications Existantes

1.5.1 Bricoraml

Plateforme de mise en relation pour services à domicile, sans composant IA. Pertinente comme référence pour la gestion des rôles et des sessions utilisateurs, mais n'adresse pas la problématique du diagnostic assisté.

1.5.2 Exemples de systèmes IA médicaux

Des outils académiques comme *CheXNet* [10] (Stanford, 2017) ont montré qu'un réseau de neurones convolutif (CNN) peut détecter la pneumonie avec une précision comparable à celle d'un radiologue. Ces systèmes nécessitent cependant une infrastructure GPU importante, inaccessible dans un contexte de projet annuel de Licence.

1.5.3 Fiverr

Marketplace de services numériques. Référence architecturale pour la gestion multi-rôles et la traçabilité des actions utilisateurs.

1.5.4 Outils académiques open source

Des librairies comme *scikit-learn* [14], *scikit-image* et *torchvision* permettent de comparer descripteurs classiques et embeddings CNN dans un cadre expérimental reproductible.

1.5.5 TaskRabbit

Application de mise en relation de services. Référence pour l'architecture de gestion des sessions de travail et l'isolation des données par session.

1.5.6 Exemples de datasets médicaux publics

Le dataset *Chest X-Ray Images (Pneumonia)* [7] disponible sur Kaggle (5 863 images, 2 classes) constitue la base d'entraînement du modèle. Son utilisation dans un cadre académique est libre de droits.

1.5.7 Maquettes

Les maquettes de l'interface ont été conçues avant le développement afin de valider le workflow utilisateur : connexion, ouverture de session, dépôt d'image, visualisation du résultat, consultation de la file de triage.

1.5.8 Comparaison

Table 1.1: Comparaison des solutions existantes

Solution	**IA**	**Open Source**	**Deployable**	**Limite**
CheXNet [10]	CNN	Non	Non (GPU)	Certification requise
Qure.ai	Deep Learning	Non	Non (SaaS)	Coût prohibitif
Aidoc	Deep Learning	Non	Non (SaaS)	Dispositif médical
PneumoScan	EfficientNet-B0+SVM	Oui	Oui (Docker)	Prototype académique

1.5.9 Conclusion

Les solutions commerciales sont performantes mais inaccessibles dans un contexte académique. PneumoScan se positionne comme un prototype déployable mettant l'accent sur la qualité architecturale, la sécurité et la traçabilité.

1.6 Méthodologie de Développement

1.6.1 Unified Process (UP)

Le projet a été conduit selon une adaptation légère de l'Unified Process (UP), méthode itérative et incrémentale particulièrement adaptée aux projets combinant développement logiciel et intégration de composants IA.

Table 1.2: Phases UP appliquées au projet

Phase	Durée estimée	Livrables
Inception	2 semaines	Étude existant, choix stack, répartition binôme
Élaboration	4 semaines	UML, schéma BDD, maquettes, binôme entraîne modèle
Construction	8 semaines	API FastAPI, Vue.js, intégration modèle, tests
Transition	2 semaines	Docker, rapport, corrections finales

1.6.2 Les types de diagrammes UML

Plusieurs types de diagrammes Unified Modeling Language (UML) ont été utilisés pour modéliser le système :

- **Diagramme de cas d'utilisation :** identification des acteurs (Radiologue, Administrateur) et de leurs interactions.
- **Diagrammes de séquences :** flux d'exécution pour chaque cas d'utilisation (connexion, analyse, triage, etc.).
- **Diagramme de classes :** structure statique des entités du domaine (Prediction, Session, Utilisateur).
- **Modèle relationnel :** schéma de la base SQLite avec tables, clés primaires, clés étrangères et index.

1.7 Plateforme de Développement et Stack Technique

1.7.1 Technologies Utilisées

Table 1.3: Stack technique du projet

Composant	**Technologie**	**Rôle**
Backend API	FastAPI 0.111 (Python 3.11)	Endpoints REST, orchestration du pipeline **[Mon apport]**
Frontend	Vue.js 3 (CDN)	Interface utilisateur réactive **[Mon apport]**
Machine Learning	scikit-learn 1.5 + PyTorch	Pipeline EfficientNet-B0 + SVM **[Binôme]**
Vision par ordinateur	OpenCV 4.9	Prétraitement, garde OOD, heatmap **[Mon apport]**
Base de données	SQLite 3	Persistance prédictions et sessions **[Mon apport]**
Sécurité	python-jose, bcrypt	JWT HS256, hachage mots de passe **[Mon apport]**
Tests	pytest 8.2	32 tests unitaires **[Mon apport]**
Déploiement	Docker Compose	Conteneurisation multi-services **[Mon apport]**
Graphiques	Chart.js 4	Tableau de bord BI **[Mon apport]**
PDF	jsPDF 2.5	Génération de rapports **[Mon apport]**

1.7.2 Choix Techniques

- **FastAPI vs Flask :** FastAPI offre la validation automatique des types (Pydantic), la documentation Swagger intégrée et de meilleures performances asynchrones, sans surcoût de configuration.
- **Vue.js 3 (CDN) :** déploiement en fichier HTML unique, sans étape de build, simplifiant le conteneur Docker frontend.
- **SQLite vs PostgreSQL :** SQLite est suffisant pour un prototype mono-instance ; la migration vers PostgreSQL est prévue en production.
- **EfficientNet-B0 + SVM :** choix retenu après comparaison expérimentale, car il offre le meilleur compromis performance/robustesse hors cas suspects de fuite.

Justification du modèle retenu

Le rapport expérimental du binôme montre des scores parfaits pour la concaténation descripteurs (100%), susceptibles d'indiquer une fuite de données. Pour un choix robuste, nous retenons **EfficientNet-B0 + SVM** : **Accuracy 86.2%** et **F1-score 88.4%** sur test, supérieurs aux approches classiques individuelles et cohérents avec la conclusion scientifique.

Table 1.4: Synthèse des approches candidates (jeu de test)

Approche	**Accuracy**	**F1**	**Décision**
HOG + LR	84.1%	88.4%	Baseline classique
ResNet50 + SVM	80.1%	85.6%	Correct mais inférieur
EfficientNet-B0 + SVM	**86.2%**	**88.4%**	**Modèle retenu**
Concaténation (Haralick+HOG+LBP) + LR	100%	100%	Résultat suspect (risque de fuite)

1.8 Objectifs et Périmètre de l'Application

Table 1.5: Périmètre fonctionnel du système

Dans le périmètre	**Hors périmètre**
Analyse de radiographies JPEG/PNG/DICOM (.dcm)	Intégration PACS hospitalier
Triage automatique 3 niveaux	Certification médicale CE/FDA
Authentification JWT multi-rôles	Intégration PACS hospitalier
Tableau de bord BI	Apprentissage en ligne
Anonymisation RGPD (masquage)	Télémédecine temps réel
Rapport PDF structuré	Déploiement multi-établissements
Déploiement Docker Compose	Déploiement multi-établissements

1.9 Conclusion du Chapitre

Ce premier chapitre a posé le cadre du projet : la problématique clinique, l'analyse des solutions existantes, la méthodologie UP et la stack technique. Le chapitre suivant détaille l'analyse et la spécification des besoins fonctionnels et non fonctionnels du système.

2. Analyse et Spécification des Besoins

2.1 Introduction

L'analyse des besoins constitue le fondement de toute conception logicielle rigoureuse. Ce chapitre identifie les acteurs du système, leurs rôles, et traduit les attentes en exigences fonctionnelles et non fonctionnelles mesurables.

2.2 Besoins Fonctionnels

2.2.1 Gestion des Utilisateurs

Table 2.1: Besoins fonctionnels — Gestion des utilisateurs

ID	Intitulé	Description	Priorité
BF-01	Connexion JWT	Authentification par identifiant/mot de passe ; retour d'un token JWT signé (HS256, TTL 8h).	**Haute**
BF-02	Contrôle des rôles	Chaque endpoint vérifie le rôle du token avant d'autoriser l'accès.	**Haute**
BF-03	Déconnexion	Suppression du token du `sessionStorage` côté client.	**Moyenne**
BF-04	Profil utilisateur	Consultation des informations de l'utilisateur courant via `GET /auth/me`.	**Faible**

2.2.2 Gestion des sessions de travail

Dans PneumoScan, la session de travail constitue l'unité opérationnelle du shift de radiologie. Elle structure la traçabilité des analyses et isole les activités d'un opérateur

sur une période donnée.

Table 2.2: Besoins fonctionnels — Gestion des sessions

ID	Intitulé	Description	Priorité
BF-05	Ouverture de session	Le radiologue ouvre une session en spécifiant le nombre de patients (1–500).	**Haute**
BF-06	Clôture explicite	Le radiologue clôture la session ; le backend met à jour le statut.	**Haute**
BF-07	Anti-zombie sessions	À chaque connexion, les sessions ouvertes précédentes du même opérateur sont automatiquement clôturées.	**Moyenne**
BF-08	Consultation des sessions	Affichage de l'historique des sessions avec compteur d'analyses effectuées.	**Faible**

2.2.3 Interaction et Communication

Table 2.3: Besoins fonctionnels — Analyse radiographique

ID	Intitulé	Description	Priorité
BF-09	Validation OOD	Vérification que l'image est une radiographie (saturation HSV < 15, densité Canny $< 35\%$). HTTP 400 sinon.	**Haute**
BF-10	Pipeline IA	Prétraitement + extraction de features (EfficientNet ou HOG fallback) + classification SVM + triage automatique.	**Haute**
BF-11	Carte d'attention HOG	Superposition colorisée (palette JET) montrant les zones contribuant à la prédiction.	**Moyenne**
BF-12	Résultat en temps réel	Le résultat (label, score, triage) est retourné et affiché immédiatement après analyse.	**Haute**

2.2.4 Priorisation des cas critiques

Dans PneumoScan : **mise en valeur des cas critiques** dans l'interface. Les cas CRITICAL apparaissent en tête de file avec badge rouge et alerte visuelle distincte, garantissant une attention immédiate du radiologue.

2.2.5 Recherche et Découverte

L'historique des analyses est filtrable par :

- Plage de dates (date de début, date de fin)
- Identifiant de session
- Niveau de triage (Critical / Moderate / Routine)

2.2.6 Anonymisation des images

Non applicable au domaine médical dans ce projet. Fonctionnalité remplacée par la **gestion de l'anonymisation** : l'utilisateur peut activer le masquage des coins PACS avant l'analyse pour supprimer les données nominatives.

2.2.7 Système d'Évaluation et de Réputation

Le **tableau de bord BI** fournit les indicateurs clés de performance :

- Taux de positivité global (cas pneumonie / total analyses)
- Volume quotidien sur les 14 derniers jours
- Répartition par niveau de triage (graphique donut)
- Estimation des heures économisées (base : 5 min par analyse)

2.2.8 Interfaces de l'application (captures réelles)

Cette section présente les écrans réellement implémentés dans PneumoScan. Chaque capture illustre un besoin fonctionnel et montre la cohérence entre spécification, implémentation et usage clinique.

Authentification et démarrage d'une session clinique

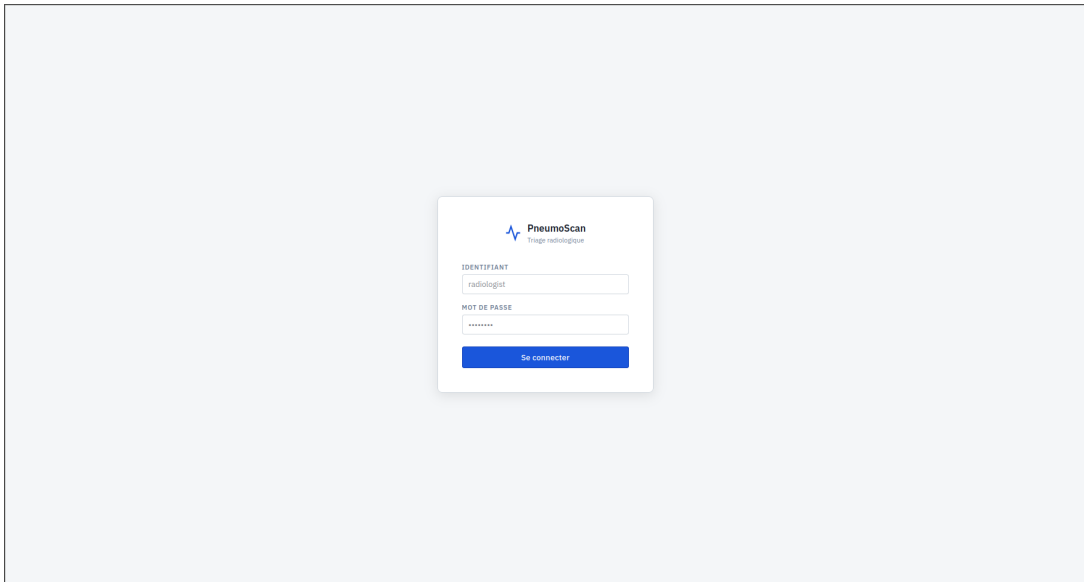


Figure 2.1: Écran de connexion (authentification JWT)

L'écran de connexion matérialise le besoin **BF-01** : l'utilisateur saisit ses identifiants, le backend valide les credentials, puis retourne un token JWT signé qui servira à l'ensemble des appels protégés.

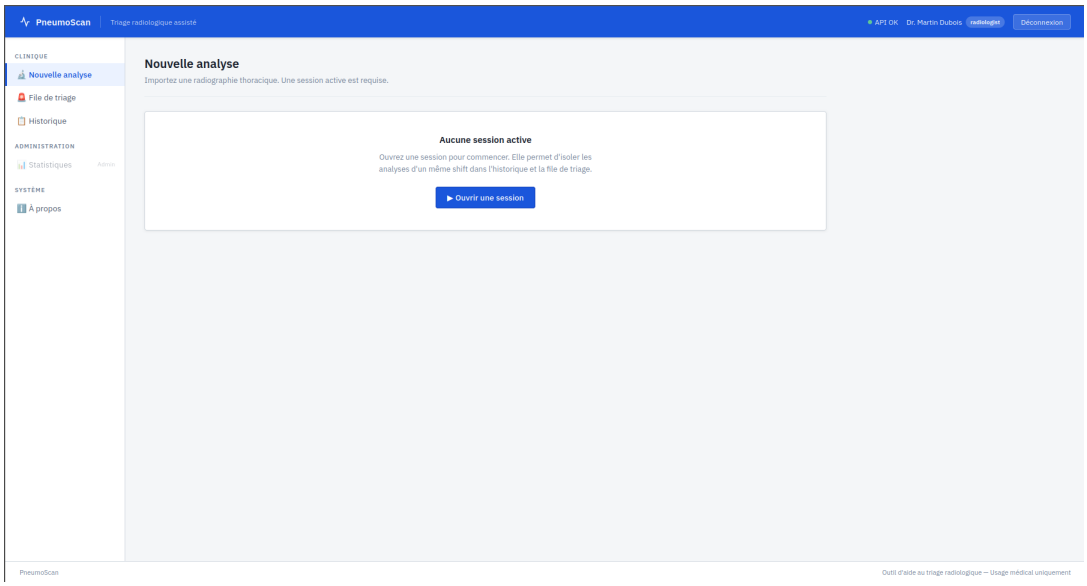


Figure 2.2: Nouvelle analyse sans session active

Cette vue montre la contrainte opérationnelle de traçabilité : sans session active, l'analyse est bloquée. Cela garantit que chaque prédiction est rattachée à un shift et à un opérateur, conformément aux besoins **BF-05** à **BF-08**.

Analyse radiographique et explicabilité

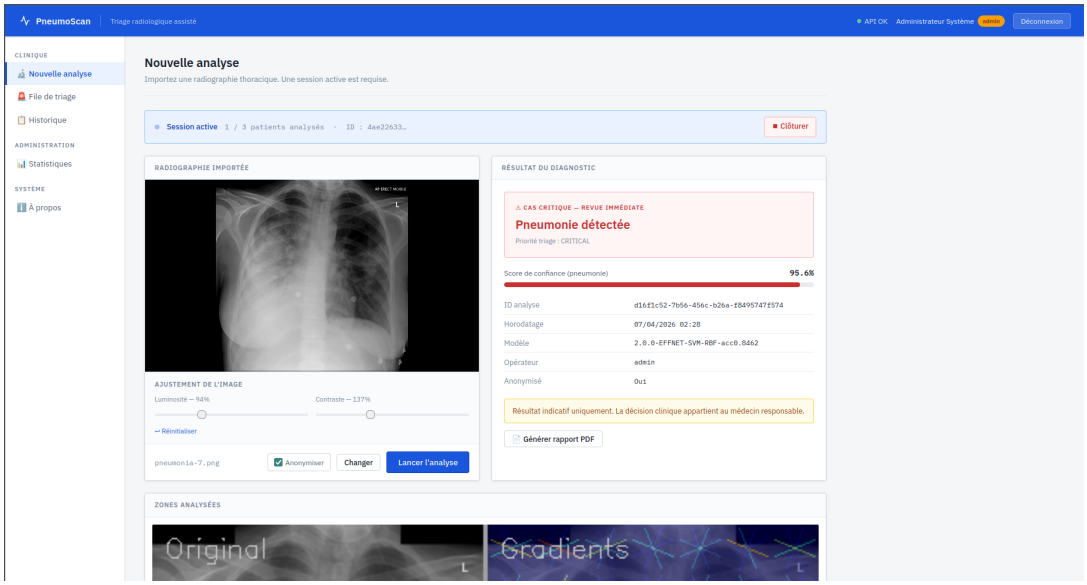


Figure 2.3: Écran d'analyse : image importée, résultat, score et triage

Cette interface regroupe le coeur métier : import de la radiographie, anonymisation optionnelle, lancement du pipeline IA, affichage du label, du score de confiance et du niveau de triage. Elle couvre les besoins **BF-09** à **BF-12**.

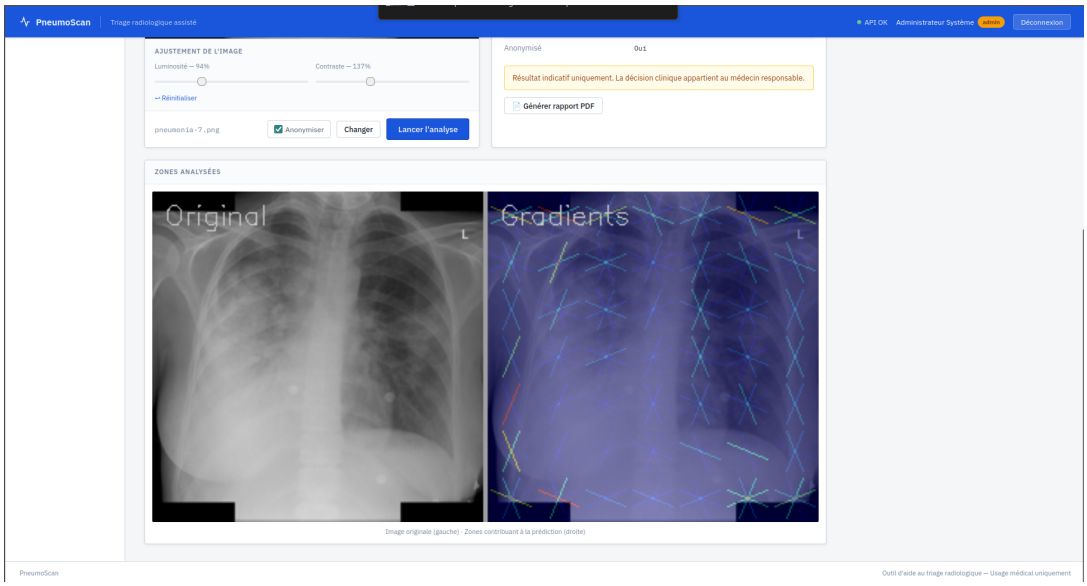
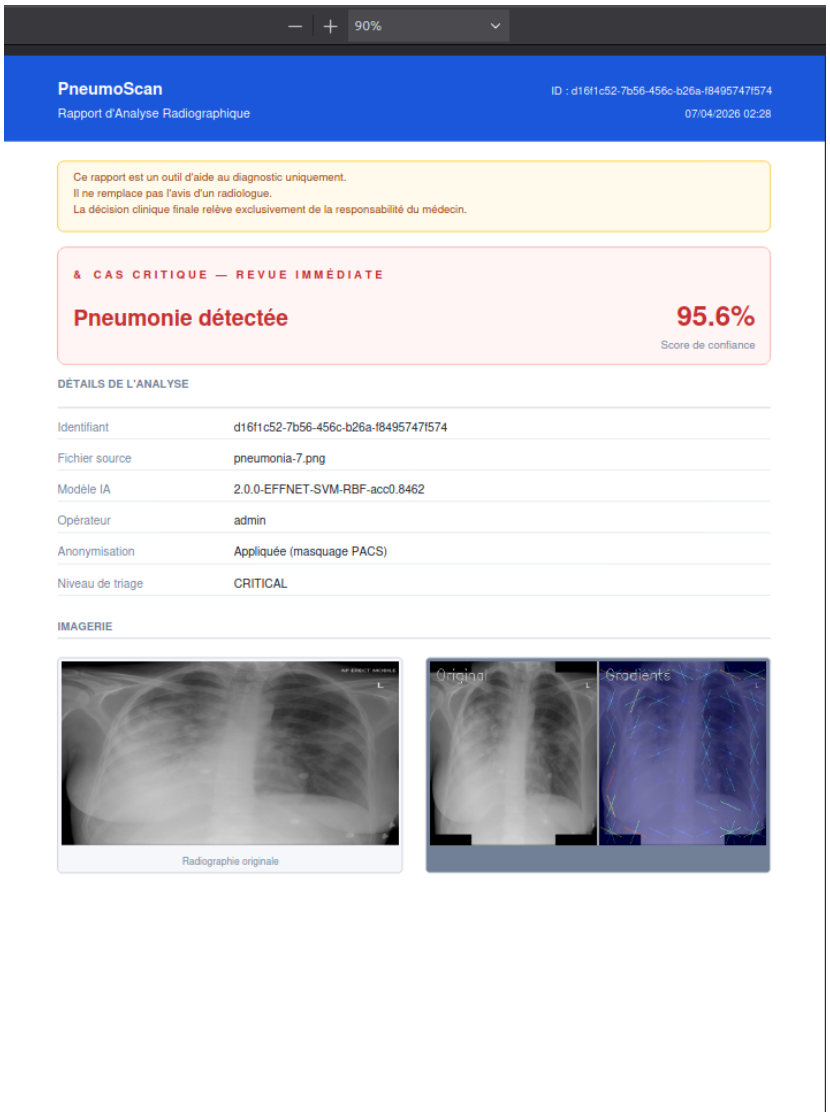


Figure 2.4: Zones analysées : comparaison image originale / gradients contributifs

La vue “Zones analysées” apporte une forme d’explicabilité visuelle : les motifs de gradients les plus contributifs sont superposés à l’image, ce qui facilite l’interprétation humaine et la discussion pédagogique du résultat.



90%

PneumoScan
Rapport d'Analyse Radiographique

ID : d16f1c52-7b56-456c-b26a-f8495747f574
07/04/2026 02:28

Ce rapport est un outil d'aide au diagnostic uniquement.
Il ne remplace pas l'avis d'un radiologue.
La décision clinique finale relève exclusivement de la responsabilité du médecin.

& CAS CRITIQUE — REVUE IMMÉDIATE

Pneumonie détectée **95.6%**
Score de confiance

DÉTAILS DE L'ANALYSE

Identifiant	d16f1c52-7b56-456c-b26a-f8495747f574
Fichier source	pneumonia-7.png
Modèle IA	2.0.0-EFFNET-SVM-RBF-acc0.8462
Opérateur	admin
Anonymisation	Appliquée (masquage PACS)
Niveau de triage	CRITICAL

IMAGERIE

Radiographie originale

Figure 2.5: Rapport PDF généré automatiquement après analyse

Le rapport PDF formalise la sortie de l'analyse : identité technique, récapitulatif du résultat, niveau de triage, images source et zones contributives. Il constitue une trace exportable et consultable hors application.

Pilotage clinique : file de triage, historique et supervision

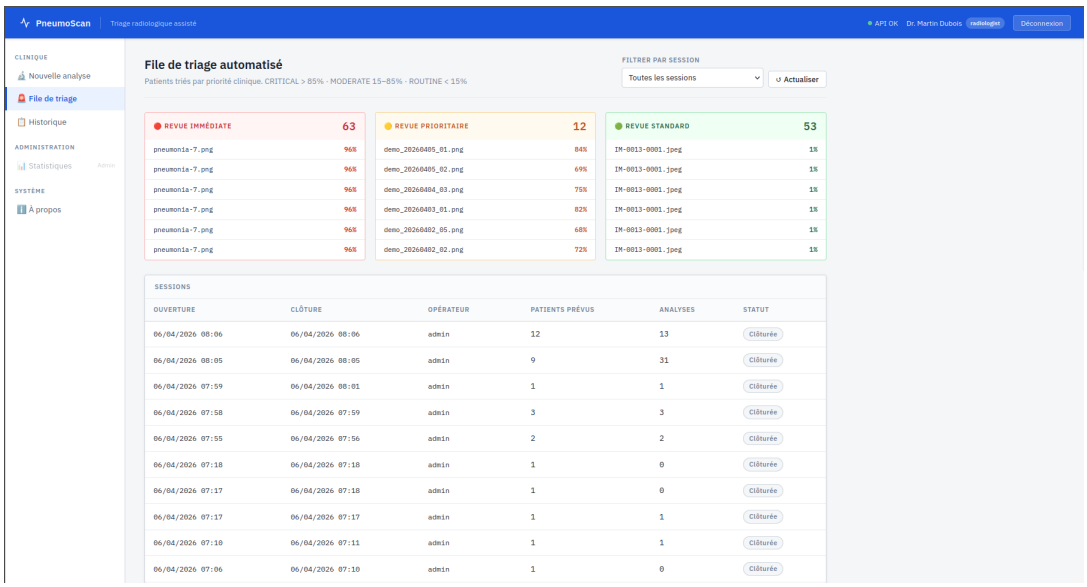


Figure 2.6: File de triage avec regroupement CRITICAL / MODERATE / ROUTINE

La file de triage met en oeuvre la priorisation clinique : les cas critiques sont exposés en tête avec une mise en forme d'alerte, les cas modérés sont orientés en revue prioritaire, et les cas routiniers restent en revue standard.

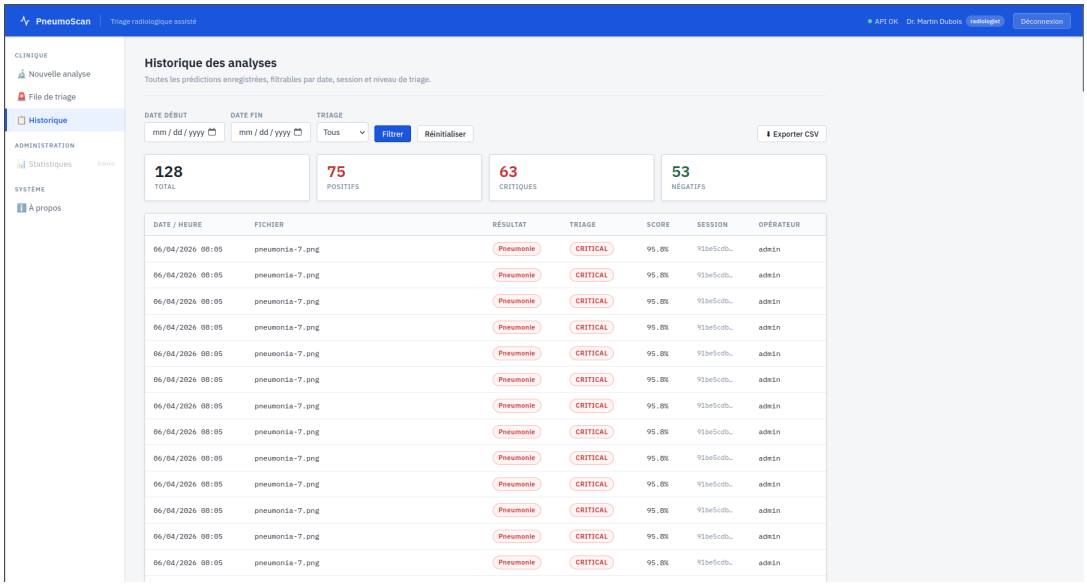


Figure 2.7: Historique filtrable des analyses et export CSV

L'historique assure la traçabilité temporelle complète : filtrage par période et niveau de triage, consultation des scores, et export CSV pour exploitation ultérieure.

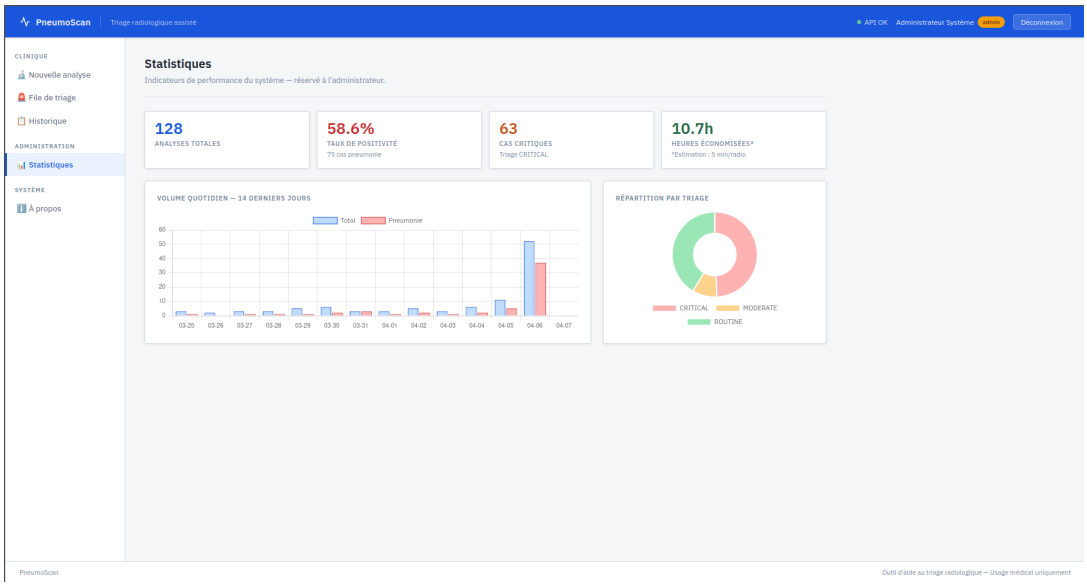


Figure 2.8: Tableau de bord administrateur (KPIs et graphiques BI)

Cette vue est réservée au rôle `admin`. Elle agrège les indicateurs de performance du service : volume, positivité, proportion de cas critiques, et estimation des heures économisées.

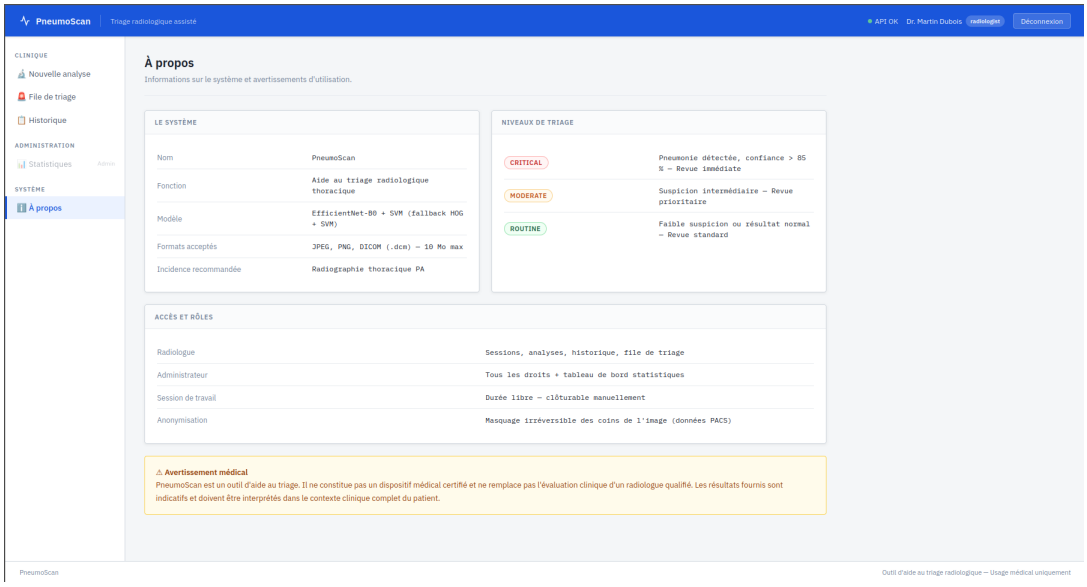


Figure 2.9: Page “À propos” : périmètre d’usage, niveaux de triage et avertissement médical

La page “À propos” rappelle le périmètre du prototype, les formats supportés, les rôles applicatifs et l’avertissement médical. Elle joue un rôle essentiel de transparence et de bon usage.

2.3 Analyse Non Fonctionnelle

Table 2.4: Besoins non fonctionnels

ID	Catégorie	Exigence
BNF-01	Performance	Temps de réponse de `/predict` inférieur à 3 secondes pour une image standard.
BNF-02	Sécurité	Tokens JWT signés HS256 via variable d'environnement. Mots de passe hachés bcrypt.
BNF-03	Disponibilité	Endpoint `GET /health` + healthcheck Docker (intervalle 30s).
BNF-04	Maintenabilité	Docstrings exhaustifs sur tous les modules. 32 tests unitaires pytest (5 classes).
BNF-05	Portabilité	Déploiement Docker Compose reproductible en une commande.
BNF-06	Conformité RGPD	Anonymisation par masquage noir irréversible. Aucune donnée nominative stockée.
BNF-07	Accessibilité	Interface responsive (mobile/desktop). Contraste WCAG AA respecté.

2.4 Acteurs et Rôles

Table 2.5: Acteurs du système et leurs responsabilités

Acteur	Rôle JWT	Responsabilités
Radiologue	`radiologist`	Crée les sessions, soumet les radiographies, consulte la file de triage et l'historique
Administrateur	`admin`	Tous les droits du radiologue + accès exclusif au tableau de bord BI
Système IA	(interne)	Exécute le pipeline de validation OOD, extraction de features et classification SVM

2.4.1 Rôle de supervision (administrateur)

Dans le contexte de PneumoScan, l'**administrateur** joue le rôle de supervision : il accède aux statistiques globales du service de radiologie, suit les indicateurs de performance et peut consulter l'ensemble des sessions de tous les opérateurs.

2.4.2 Rôle opérationnel (radiologue)

Le **radiologue** est l'utilisateur principal du système. Il réalise les analyses, gère ses sessions de travail et consulte la file de triage pour optimiser son workflow de lecture.

2.4.3 Administrateur

L'administrateur dispose de l'ensemble des droits du radiologue, augmentés de l'accès au tableau de bord BI. Il est identifiable par le token JWT portant le rôle `admin`.

2.5 Acteurs et Cas d'Utilisation

2.5.1 Diagramme de cas d'utilisation

Le diagramme ci-dessous représente les cas d'utilisation principaux du système :

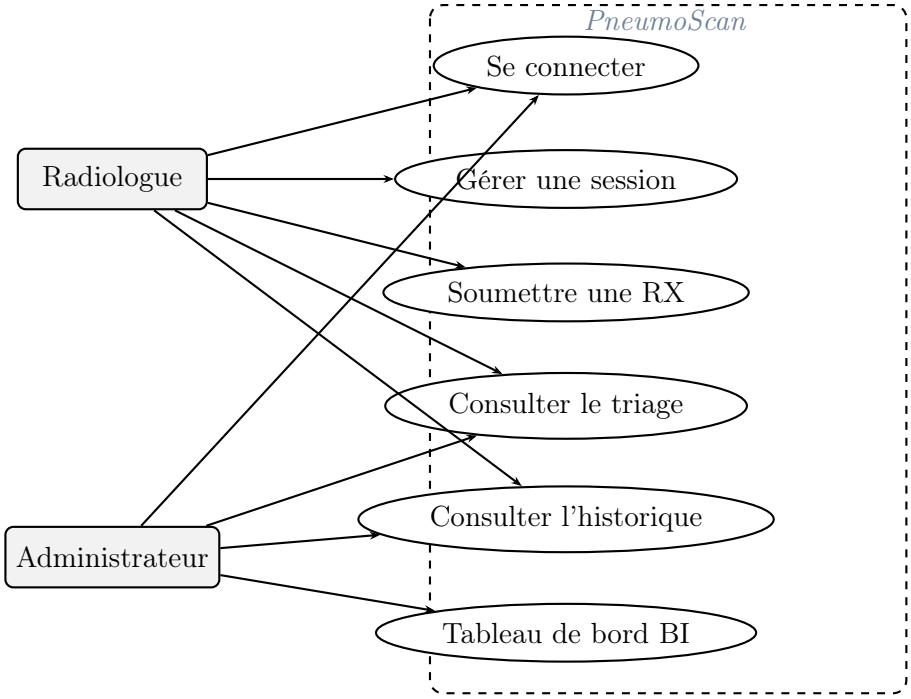


Figure 2.10: Diagramme de cas d'utilisation global

2.6 Description des Cas d'Utilisation

2.6.1 Pour les Utilisateurs

Se connecter / Se déconnecter

Table 2.6: CU-01 — Se connecter

Acteur principal	Radiologue / Administrateur
Précondition	L'utilisateur possède un compte valide
Scénario nominal	1. L'utilisateur saisit son identifiant et mot de passe
	2. Le backend valide les identifiants et génère un token JWT
	3. Le frontend stocke le token dans le `sessionStorage`
	4. L'interface principale est affichée
Scénario alternatif	Identifiants incorrects → message d'erreur inline
Postcondition	Token JWT valide, sessions fantômes clôturées

2.6.2 Publication, Recherche et Interaction

Soumettre une radiographie pour analyse

Table 2.7: CU-02 — Soumettre une radiographie

Acteur principal	Radiologue
Précondition	Session active ouverte, token JWT valide
Scénario nominal	1. Le radiologue dépose un fichier JPEG/PNG/DICOM (.dcm)
	2. Le frontend prévisualise le cliché
	3. Le radiologue clique sur "Lancer l'analyse"
	4. L'API valide le format et applique la garde OOD
	5. Le pipeline EfficientNet-B0+SVM est exécuté (fallback HOG+SVM si artefact absent)
	6. Le résultat (label, score, niveau de triage) est retourné
	7. La carte d'attention HOG est générée et affichée
	8. La prédiction est persistée en base de données
Exception	Image non radiographique → HTTP 400 + message d'erreur inline
Postcondition	Prédiction stockée, file de triage mise à jour

2.6.3 Paiement et Évaluation

Dans PneumoScan, cette dimension correspond à l'**évaluation des performances** du système via le tableau de bord BI et la génération de rapports PDF, qui constituent la trace formelle de chaque analyse.

2.7 Diagramme de cas d'utilisation

Voir figure 2.1 en section 2.5.

2.8 Conclusion du Chapitre

Ce chapitre a défini les acteurs, les besoins fonctionnels et non fonctionnels, et les principaux cas d'utilisation du système. Le chapitre suivant aborde la conception et la réalisation technique de PneumoScan.

3. Conception et Réalisation

3.1 Introduction

Ce chapitre présente la conception détaillée et la réalisation de PneumoScan. Il suit la structure UML définie en phase d'élaboration et documente les choix d'implémentation de chaque composant.

3.2 Architecture Logicielle Refactorisée (Clean Code & SOLID)

Afin d'améliorer la maintenabilité, la robustesse et la lisibilité du projet en vue d'une soutenance technique, une refactorisation architecturale a été réalisée selon une séparation explicite des responsabilités.

Table 3.1: Organisation modulaire du backend après refactorisation

Couche	**Fichiers**	**Responsabilité principale**
Contrôleurs HTTP	`main.py`	Déclaration des routes FastAPI, validation des entrées HTTP, mapping des exceptions vers les codes d'erreur API
Services applicatifs	`services.py`	Orchestration des cas d'usage (analyse, sessions, historique, dashboard) sans dépendance au transport HTTP
Configuration	`settings.py`	Centralisation des paramètres d'exécution (version, répertoire heatmaps, CORS)
Sécurité	`auth.py`	Authentification JWT, RBAC, dépendances FastAPI de contrôle d'accès
Données	`database.py`	Persistance SQLite, règles de triage, agrégations BI, gestion des sessions
ML & image	`predictor.py`	Validation OOD, prétraitement, extraction HOG, inférence SVM, génération heatmap, anonymisation
Validation	`utils.py`	Validation uniforme des métadonnées d'upload et des contraintes de taille

Justification SOLID

- **S (Single Responsibility):** chaque module a un rôle unique (ex. `services.py` n'implémente pas de logique SQL directe).
- **O (Open/Closed):** l'ajout d'une nouvelle stratégie d'analyse peut être fait dans la couche service sans modifier les routes existantes.
- **L (Liskov):** les fonctions de service exposent des contrats stables consommés par les contrôleurs.
- **I (Interface Segregation):** les dépendances FastAPI utilisent des interfaces de rôle dédiées (`require_radiologist`, `require_admin`).
- **D (Dependency Inversion):** `main.py` dépend d'abstractions de service (cas d'usage), et non de détails de persistance ou de traitement image.

3.3 Diagrammes des Séquences

3.3.1 Connexion

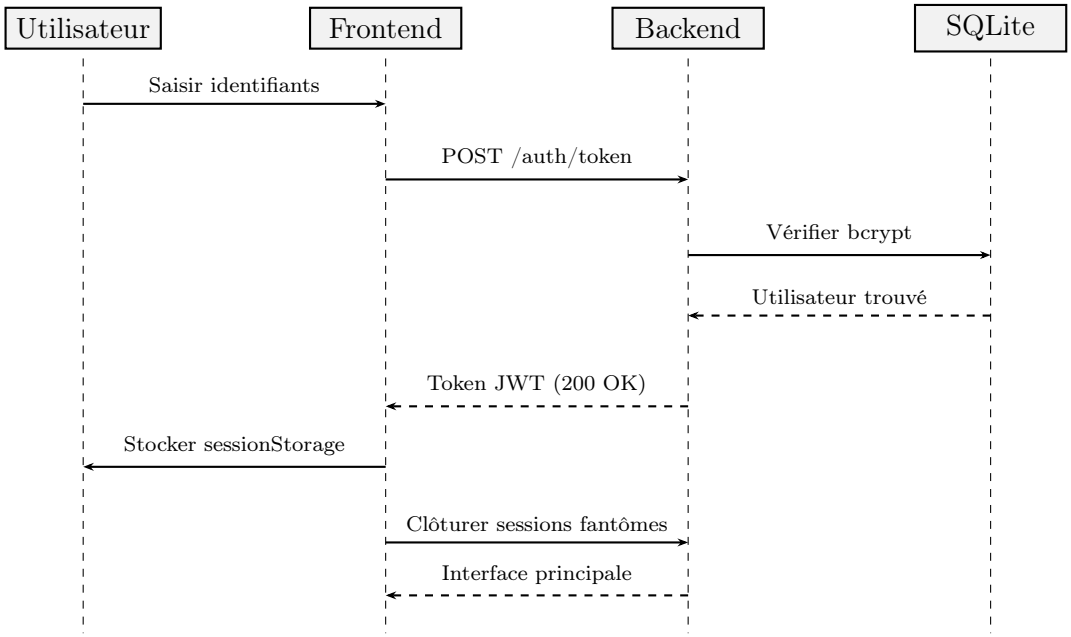


Figure 3.1: Diagramme de séquence — Connexion

3.3.2 Inscription

L'inscription n'est pas disponible en mode libre dans ce prototype. Les comptes sont pré-configurés en mémoire (phase prototype). En production, un endpoint `POST /auth/register` permettrait la création de comptes avec validation par un administrateur.

3.3.3 Soumettre une radiographie

Dans le contexte de PneumoScan : **soumettre une radiographie pour analyse**. Voir le diagramme de séquence détaillé ci-dessous.

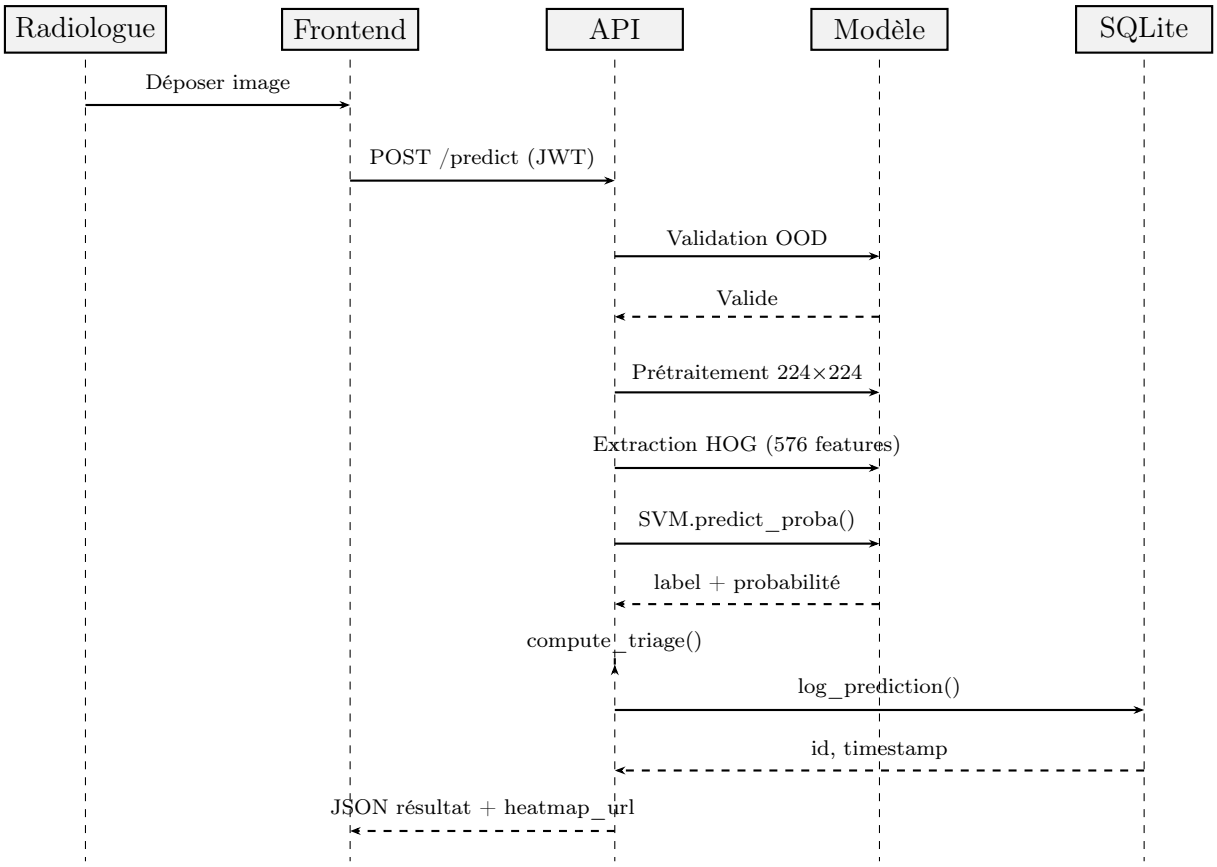


Figure 3.2: Diagramme de séquence — Analyse radiographique

3.3.4 Consulter mes activités

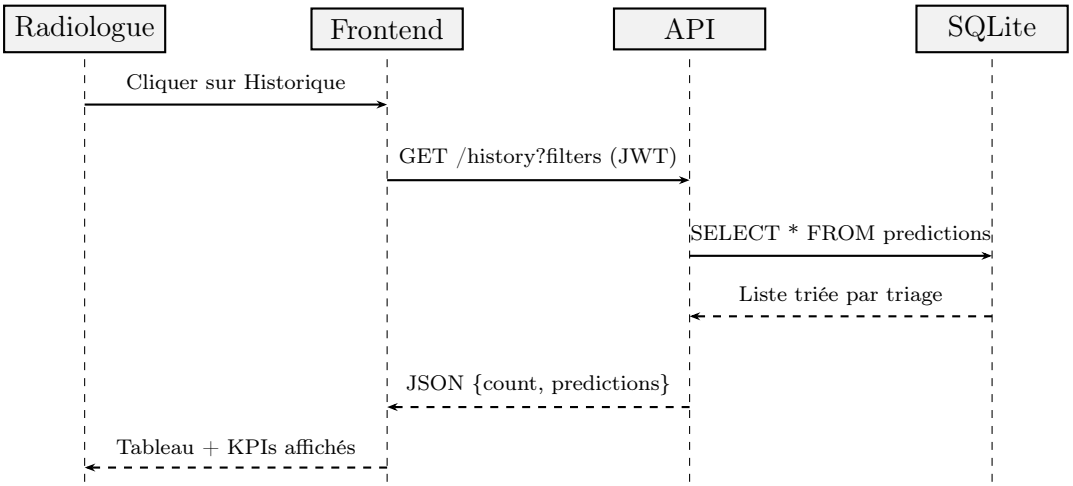


Figure 3.3: Diagramme de séquence — Consultation de l'historique

3.3.5 Ouvrir une session de travail

Dans PneumoScan : **ouvrir une session de travail**.

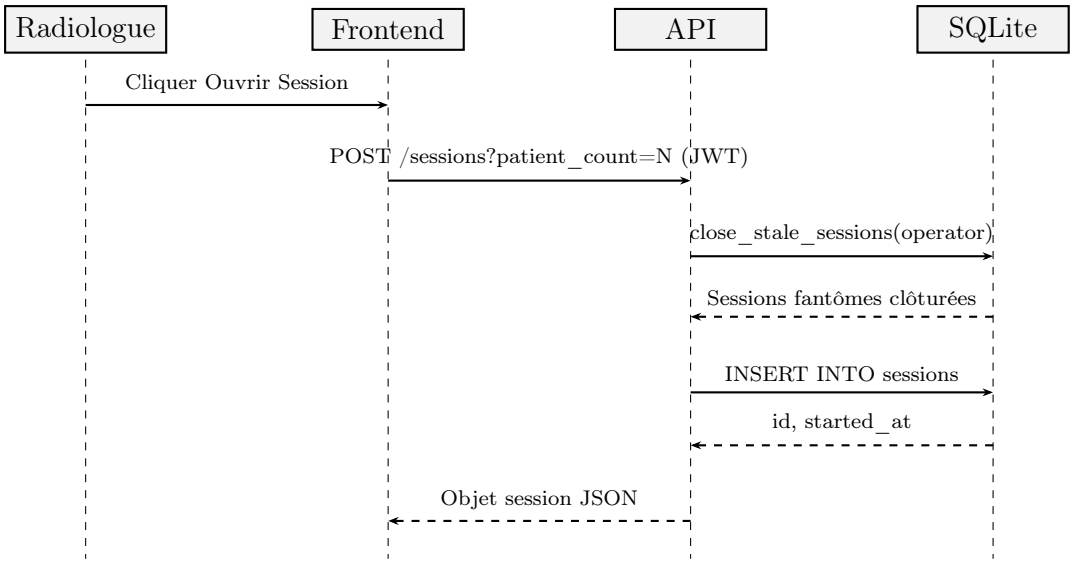


Figure 3.4: Diagramme de séquence — Ouverture de session

3.3.6 Consulter la file de triage

Dans PneumoScan : **consulter la file de triage** avec filtres de session.

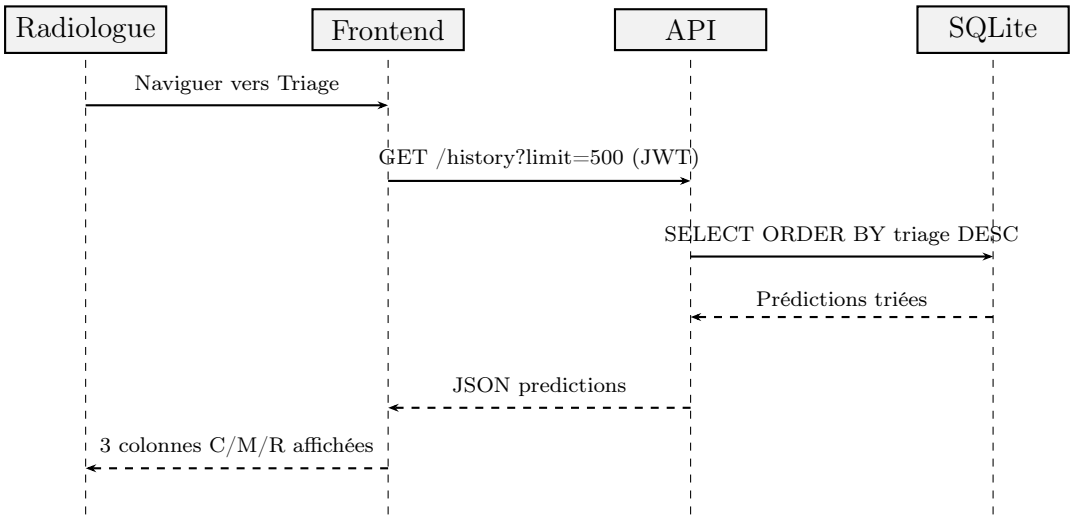


Figure 3.5: Diagramme de séquence — Consultation du triage

3.3.7 Consulter le tableau de bord BI

Dans PneumoScan : **tableau de bord BI** (admin uniquement).

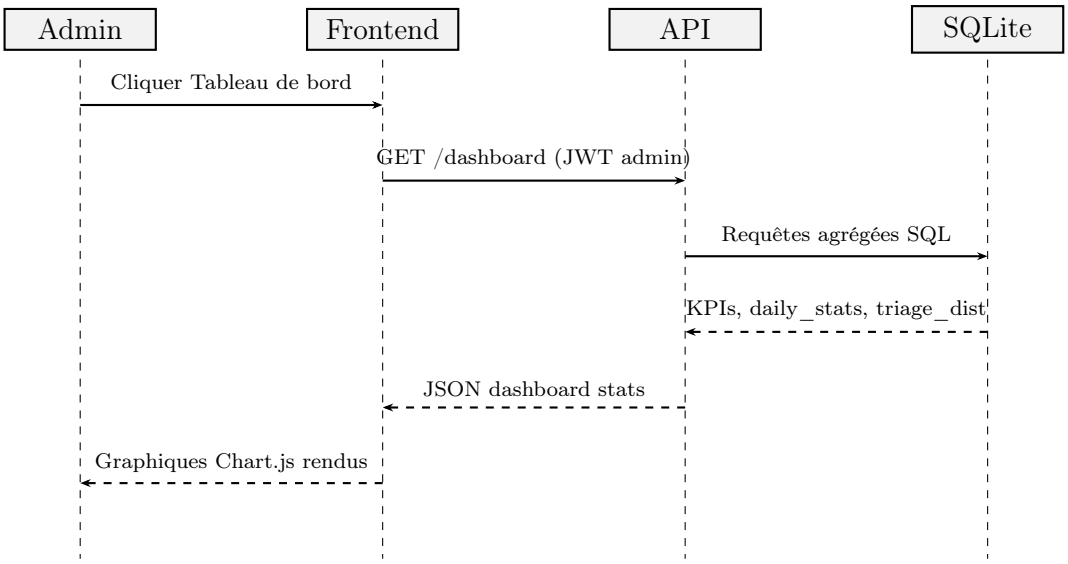


Figure 3.6: Diagramme de séquence — Tableau de bord BI

3.3.8 Générer un rapport PDF

Dans PneumoScan : **anonymisation et génération du rapport PDF**. Le radiologue active l'anonymisation avant analyse, puis génère le rapport PDF qui constitue la trace formelle de l'analyse.

3.3.9 Diagramme de classe

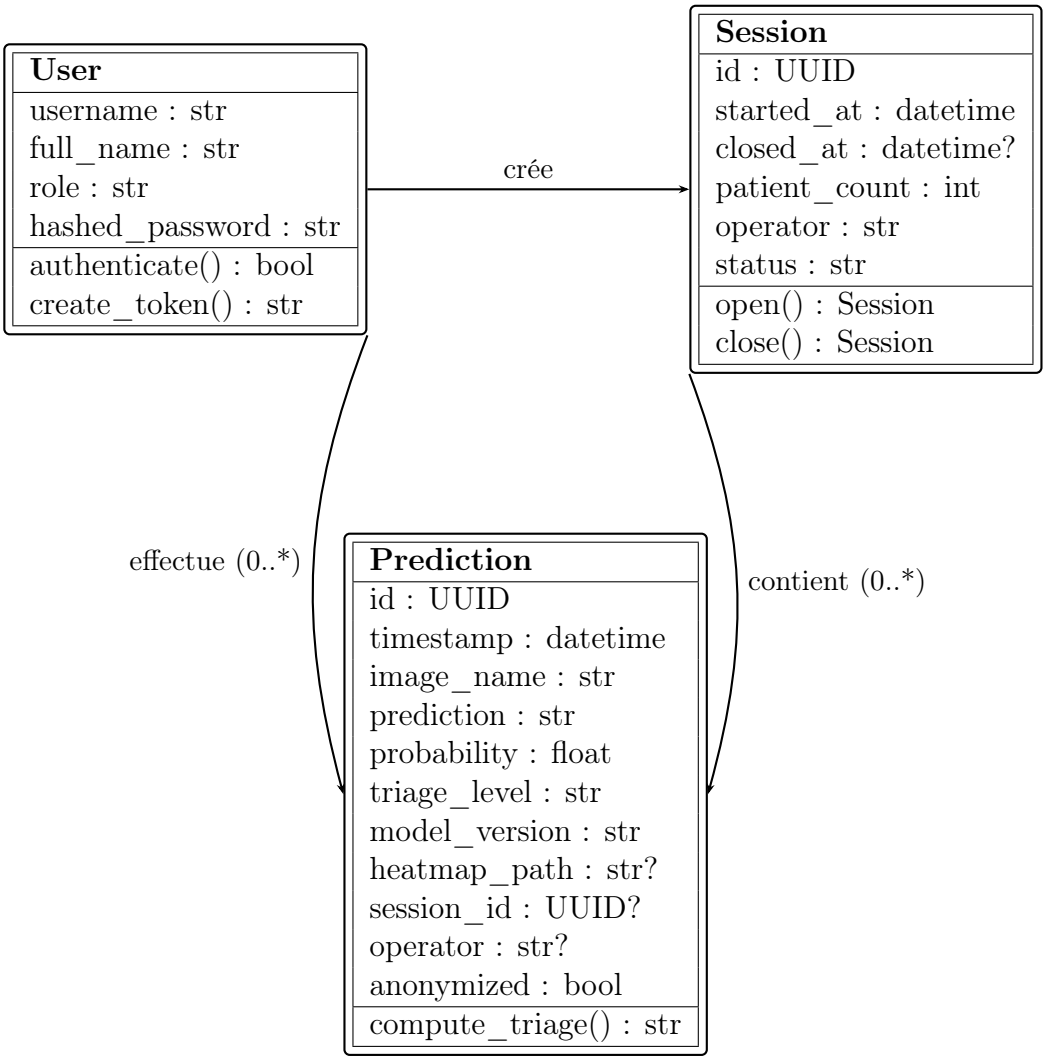


Figure 3.7: Diagramme de classes simplifié

3.3.10 Dictionnaire des données

Table 3.2: Dictionnaire des données — Table `predictions`

Attribut	**Type**	**Contrainte**	**Description**
`id`	TEXT	PK, NOT NULL	Identifiant UUID v4
`timestamp`	TEXT	NOT NULL	Horodatage ISO 8601 UTC
`image_name`	TEXT	NOT NULL	Nom du fichier soumis
`prediction`	TEXT	NOT NULL	`PNEUMONIA` ou `NORMAL`
`probability`	REAL	NOT NULL	Score de confiance $[0.0, 1.0]$
`model_version`	TEXT	NOT NULL	Version du modèle utilisé
`heatmap_path`	TEXT	NULL OK	Chemin vers la heatmap HOG
`triage_level`	TEXT	NOT NULL	`CRITICAL`, `MODERATE` ou `ROUTINE`
`session_id`	TEXT	NULL OK, FK	Référence vers `sessions.id`
`operator`	TEXT	NULL OK	Nom d'utilisateur de l'opérateur
`anonymized`	INTEGER	NOT NULL, Def=0	1 si anonymisée, 0 sinon

Table 3.3: Dictionnaire des données — Table `sessions`

Attribut	Type	Contrainte	Description
`id`	TEXT	PK, NOT NULL	Identifiant UUID v4
`started_at`	TEXT	NOT NULL	Horodatage d'ouverture ISO 8601
`closed_at`	TEXT	NULL OK	Horodatage de clôture (NULL si ouverte)
`patient_count`	INTEGER	NOT NULL	Nombre de patients prévus
`operator`	TEXT	NOT NULL	Nom d'utilisateur du radiologue
`status`	TEXT	NOT NULL, Def=open	`open` ou `closed`

3.3.11 Le modèle relationnel

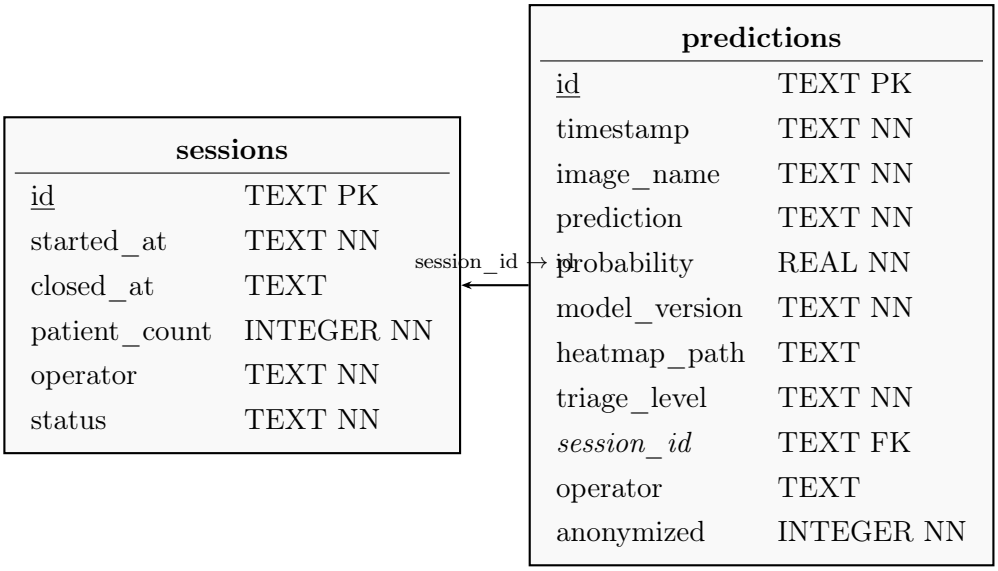


Figure 3.8: Modèle relationnel — Base de données SQLite

3.3.12 Code SQL

Listing 3.1: Schéma SQL complet de la base de données

```
-- Table des prédictions
CREATE TABLE IF NOT EXISTS predictions (
```

```sql
    id            TEXT PRIMARY KEY,   -- UUID v4
    timestamp     TEXT NOT NULL,      -- ISO 8601 UTC
    image_name    TEXT NOT NULL,
    prediction    TEXT NOT NULL,      -- 'PNEUMONIA' | 'NORMAL'
    probability   REAL NOT NULL,      -- Confiance [0.0, 1.0]
    model_version TEXT NOT NULL,
    heatmap_path  TEXT,
    triage_level  TEXT NOT NULL,      -- 'CRITICAL' |
       'MODERATE' | 'ROUTINE'
    session_id    TEXT,               -- FK -> sessions.id
    operator      TEXT,
    anonymized    INTEGER NOT NULL DEFAULT 0
);

-- Table des sessions de travail
CREATE TABLE IF NOT EXISTS sessions (
    id            TEXT PRIMARY KEY,
    started_at    TEXT NOT NULL,
    closed_at     TEXT,              -- NULL si session ouverte
    patient_count INTEGER NOT NULL,
    operator      TEXT NOT NULL,
    status        TEXT NOT NULL DEFAULT 'open'
);

-- Index pour les filtres fréquents
CREATE INDEX IF NOT EXISTS idx_pred_session
    ON predictions(session_id);
CREATE INDEX IF NOT EXISTS idx_pred_timestamp
    ON predictions(timestamp);
CREATE INDEX IF NOT EXISTS idx_pred_triage
    ON predictions(triage_level);
```

3.4 Outils, Langages et Environnements de Développement

3.4.1 Langages de Programmation

Table 3.4: Langages de programmation utilisés

Langage	**Version**	**Usage dans le projet**
Python	3.11	Backend API, pipeline ML, tests unitaires
JavaScript	ES2022	Frontend Vue.js 3, génération PDF
SQL	SQLite 3	Requêtes BDD, agrégations BI

3.4.2 Frameworks et Bibliothèques

Table 3.5: Frameworks et bibliothèques

Outil	**Version**	**Rôle**
FastAPI	0.111	Framework API REST asynchrone
Vue.js	3.x (CDN)	Framework frontend réactif
scikit-learn + PyTorch	1.5.0 / 2.x	Pipeline EfficientNet-B0 + SVM (**[Binôme]**)
OpenCV	4.9.0	Prétraitement images, garde OOD, heatmap
scikit-image	0.22.0	Extraction du descripteur HOG
Pillow	10.3.0	Ouverture et conversion des images
Chart.js	4.4.0	Graphiques tableau de bord BI
jsPDF	2.5.1	Génération de rapports PDF côté client
pytest	8.2.0	Tests unitaires (31 cas, 5 classes)
python-jose	3.3.0	Encodage/décodage JWT HS256
passlib	1.7.4	Hachage bcrypt des mots de passe

3.4.3 Base de Données

SQLite 3 a été retenu pour le prototype : léger, sans serveur dédié, embarqué dans Python via le module standard `sqlite3`. Le schéma comporte 2 tables et 3 index. La connexion est gérée par un gestionnaire de contexte Python (`@contextmanager`) garantissant la fermeture et le rollback en cas d'exception.

3.4.4 Communication en Temps Réel

La communication entre le frontend et le backend est réalisée par des appels **HTTP/REST** standard (fetch API). La mise à jour de la file de triage est déclenchée explicitement

par le radiologue (bouton “Actualiser”), sans connexion persistante WebSocket dans cette version prototype.

3.4.5 Déploiement et Hébergement

Point central de la démonstration

Le déploiement conteneurisé via `Docker Compose` constitue un axe majeur de la soutenance : il démontre la reproductibilité, la portabilité et la capacité d’industrialisation du prototype au-delà de la simple preuve de concept algorithmique.

Table 3.6: Configuration Docker Compose

Service	Port	Description
`pneumoscan-backend`	8000	API FastAPI — Python 3.11-slim + dépendances ML
`pneumoscan-frontend`	3000	Interface Vue.js servie par Nginx Alpine (hôte)

Listing 3.2: Déploiement en trois commandes

```
# 1. Placer le modèle entraîné (contribution du binôme)
cp pneumonia_hog_model.pkl ./backend/model/

# 2. Construire et démarrer les deux services
docker compose up --build

# 3. Accéder à l'application
# Interface : http://localhost:3000
# Documentation API : http://localhost:8000/docs
```

3.4.6 Stockage des fichiers

Les heatmaps HOG générées sont stockées sur le disque dans le répertoire `/app/heatmaps/` (volume Docker persistant). Chaque fichier est nommé `heatmap_{uuid_hex}.png` et servi statiquement via l’endpoint `GET /heatmap/{filename}`.

3.4.7 Environnements de Développement

Table 3.7: Environnement de développement

Outil	**Usage**
VS Code	Éditeur principal (extensions Python, Pylance, Docker)
Git / GitHub	Versioning, collaboration, `github.com/Anis-Aissani`
Docker Desktop	Exécution et test des conteneurs en local
Postman	Test et débogage des endpoints API
Jupyter Notebook	Exploration du dataset et entraînement du modèle (**[Binôme]**)

3.5 Description des Résultats Attendus

Table 3.8: Résultats attendus du système

Fonctionnalité	**Résultat attendu**
Analyse d'une radiographie	Label (PNEUMONIA / NORMAL), score de confiance, niveau de triage, heatmap HOG
Triage automatique	File de travail triée Critical → Moderate → Routine
Rejet OOD	HTTP 400 avec message d'erreur explicite pour toute image non radiographique
Authentification	Token JWT valide retourné en < 500ms
Rapport PDF	Document A4 structuré, grille rigide, images côte-à-côte
Tableau de bord BI	KPIs mis à jour en temps réel, graphiques Chart.js
Tests unitaires	32 tests passants en < 30 secondes
Déploiement	Application accessible sur http://localhost:3000 après `docker compose up`

Conclusion générale

PneumoScan réalise l'objectif qui lui était assigné : concevoir et développer une architecture logicielle complète intégrant un composant d'intelligence artificielle pré-entraîné dans un contexte médical réaliste.

Le cadre UP a structuré l'avancement selon la séquence suivante : Inception (2 sem.), Élaboration (4 sem.), Construction (8 sem.), Transition (2 sem.). Cette progression a permis de synchroniser la contribution modèle du binôme et ma contribution applicative jusqu'au déploiement final.

Les contributions personnelles du projet (ma partie) comprennent :

1. Un **pipeline applicatif robuste** avec garde OOD OpenCV, résolvant le problème des faux positifs sur images non radiographiques.
2. Une **architecture sécurisée** (JWT HS256, RBAC, sessions anti-zombie) respectant les bonnes pratiques de sécurité des applications web.
3. Une **interface utilisateur** Vue.js 3 responsive, avec file de triage, contrôles d'image, génération de PDF et tableau de bord BI.
4. Une **suite de tests automatisés** (32 cas pytest) couvrant les composants critiques.
5. Un **déploiement conteneurisé** reproductible via Docker Compose.

Les perspectives d'évolution prioritaires identifiées sont :

- Intégration du support **DICOM** pour les fichiers hospitaliers réels
- Fine-tuning du pipeline **EfficientNet-B0 + SVM** sur données plus diversifiées
- Implémentation de méthodes d'explicabilité (**Grad-CAM**, SHAP)
- Migration vers **PostgreSQL** pour un déploiement multi-utilisateurs

Ce projet m'a permis de mettre en œuvre l'ensemble du cycle de développement logiciel — de la conception UML au déploiement Docker — en intégrant le modèle produit par mon binôme dans l'application que j'ai construite, tout en confrontant les défis réels de l'intégration d'un composant IA dans une architecture applicative professionnelle.

Résumé

Résumé : PneumoScan est un système web d'aide au triage des radiographies thoraciques, développé dans le cadre d'un projet annuel de Licence Informatique (L3) à l'Université de Rouen Normandie. Le système intègre un pipeline de machine learning (EfficientNet-B0 + SVM, Accuracy 86.2% / F1 88.4%) dans une architecture logicielle complète : API REST FastAPI, interface Vue.js 3, base de données SQLite, authentification JWT/RBAC et déploiement Docker Compose. Une garde hors-distribution (OOD) basée sur OpenCV rejette les images non radiographiques avant toute inférence. L'interface propose une file de triage automatique (Critical / Moderate / Routine), un tableau de bord BI et la génération de rapports PDF structurés.

Mots-clés : Intelligence Artificielle, Machine Learning, HOG, SVM, Triage Médical, FastAPI, Vue.js, JWT, Docker, SQLite, Radiographie Thoracique, Pneumonie, Out-Of-Distribution.

Liste des mots-clés et leurs définitions

HOG (Histogram of Oriented Gradients) Descripteur de features locales calculant la distribution des orientations des gradients dans des cellules d'une image [2].

SVM (Support Vector Machine) Classifieur supervisé cherchant l'hyperplan de marge maximale entre deux classes [1].

JWT (JSON Web Token) Standard RFC 7519 pour transmettre des informations signées entre parties sous forme de token compact [6].

RBAC (Role-Based Access Control) Modèle de contrôle d'accès attribuant des permissions à des rôles, eux-mêmes attribués à des utilisateurs [13].

OOD (Out-Of-Distribution) Données ne ressemblant pas à celles vues à l'entraînement, pouvant provoquer des prédictions erronées à haute confiance.

PACS (Picture Archiving and Communication System) Système hospitalier de stockage et distribution des images médicales.

API REST Interface de programmation basée sur les principes REST (Representational State Transfer) définis par Fielding [3].

Docker Compose Outil d'orchestration de conteneurs Docker permettant de définir et démarrer des applications multi-services.

Triage clinique Processus médical de classification des patients selon l'urgence de leur prise en charge.

Bibliography

[1] C. Cortes, V. Vapnik. *Support-vector networks.* Machine Learning, 20(3), pp. 273–297, 1995.

[2] N. Dalal, B. Triggs. *Histograms of Oriented Gradients for Human Detection.* IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 886–893, 2005.

[3] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures.* Thèse de doctorat, Université de Californie, Irvine, 2000.

[4] I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning.* MIT Press, 2016.

[5] G. Huang, Z. Liu, L. van der Maaten, K. Q. Weinberger. *Densely Connected Convolutional Networks.* IEEE CVPR, pp. 4700–4708, 2017.

[6] M. Jones, J. Bradley, N. Sakimura. *JSON Web Token (JWT).* RFC 7519, IETF, 2015.

[7] P. Mooney. *Chest X-Ray Images (Pneumonia).* Kaggle, 2018. `https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia`

[8] Parlement européen et Conseil de l'UE. *Règlement (UE) 2017/745 relatif aux dispositifs médicaux (MDR).* Journal officiel de l'Union européenne, L 117, 2017.

[9] N. Provos, D. Mazieres. *A Future-Adaptable Password Scheme.* USENIX Annual Technical Conference, 1999.

[10] P. Rajpurkar et al. *CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning.* arXiv:1711.05225, 2017.

[11] Parlement européen et Conseil de l'UE. *Règlement (UE) 2016/679 — Protection des données à caractère personnel (RGPD).* Journal officiel de l'Union européenne, L 119, 2016.

[12] W. Samek, K.-R. Müller. *Explainable Artificial Intelligence: Understanding, Visualizing and Interpreting Deep Neural Network Predictions.* ITU Journal, vol. 1, 2019.

[13] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, C. E. Youman. *Role-Based Access Control Models.* IEEE Computer, 29(2), pp. 38–47, 1996.

[14] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, pp. 2825–2830, 2011.

[15] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* IEEE ICCV, pp. 618–626, 2017.

[16] Organisation mondiale de la Santé (OMS). *Pneumonia — Key Facts.* World Health Organization, 2022. `https://www.who.int/news-room/fact-sheets/detail/pneumonia`

[17] M. Elakef Zenagui. *Artefact modèle de classification pneumonie utilisé pour l'intégration API de PneumoScan.* Google Drive, 2026. `https://drive.google.com/file/d/1CH2zGzIfj5EJY3V_-bXOqAYCs7CbaPfN/view?usp=sharing`

Acronyms

API Application Programming Interface.

BI Business Intelligence.

HOG Histogram of Oriented Gradients.

IA Intelligence Artificielle.

JWT JSON Web Token.

OOD Out-Of-Distribution.

PACS Picture Archiving and Communication System.

RBAC Role-Based Access Control.

SVM Support Vector Machine.

UML Unified Modeling Language.

UP Unified Process.

Acronyms

API Application Programming Interface.

BI Business Intelligence.

HOG Histogram of Oriented Gradients.

IA Intelligence Artificielle.

JWT JSON Web Token.

OOD Out-Of-Distribution.

PACS Picture Archiving and Communication System.

RBAC Role-Based Access Control.

SVM Support Vector Machine.

UML Unified Modeling Language.

UP Unified Process.